Program one assignment answer

1) *Problem one – first order differential*

a) Find and plot the analytical (exact) solution to the differential equation for $0 \leq t \leq 10$.

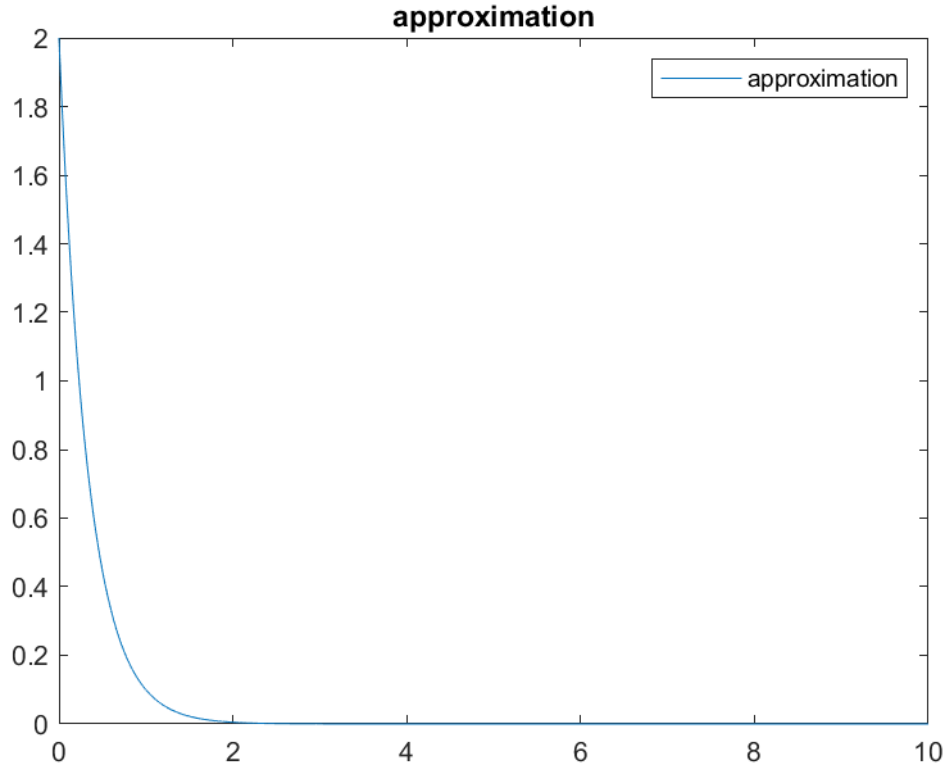


Figure 1: This is the graph of the first order differential equation using Matlab and the program to approximate the solution.

b) Write a program in C (++) to plot a numerical solution using (3). You may have to try several values of Δt to get a good enough approximation.

Work for the analytical solution will be put in appendix B and the code will be placed in appendix A.

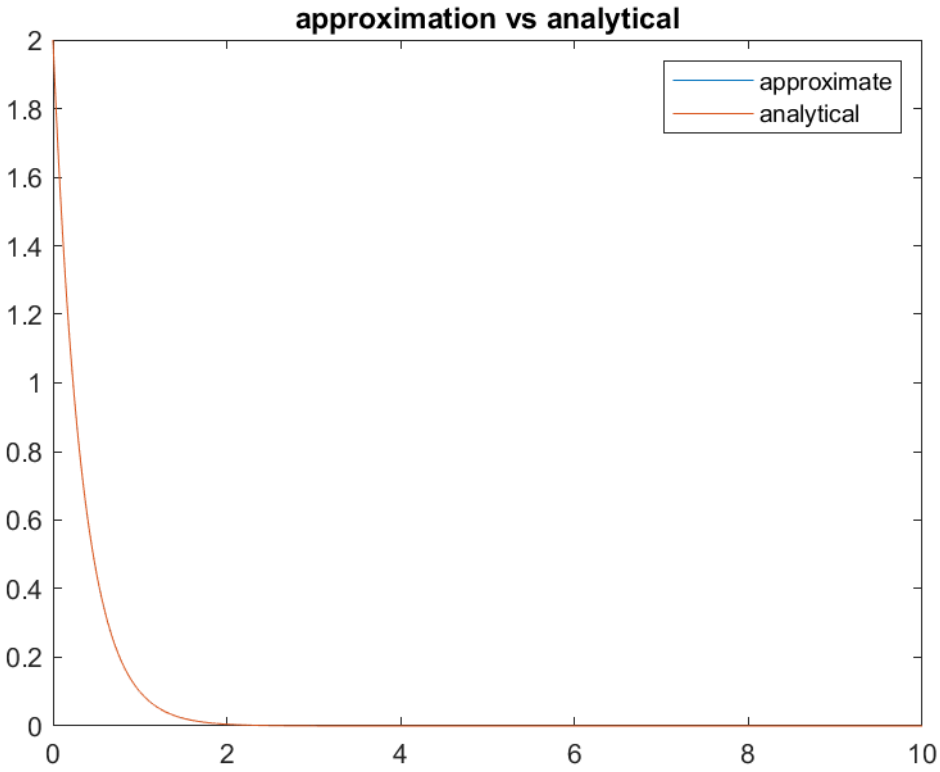


Figure 2: Using Matlab and the program this is the plot of the approximation and the analytical solution.

c) Compare the exact solution with the approximate solution.

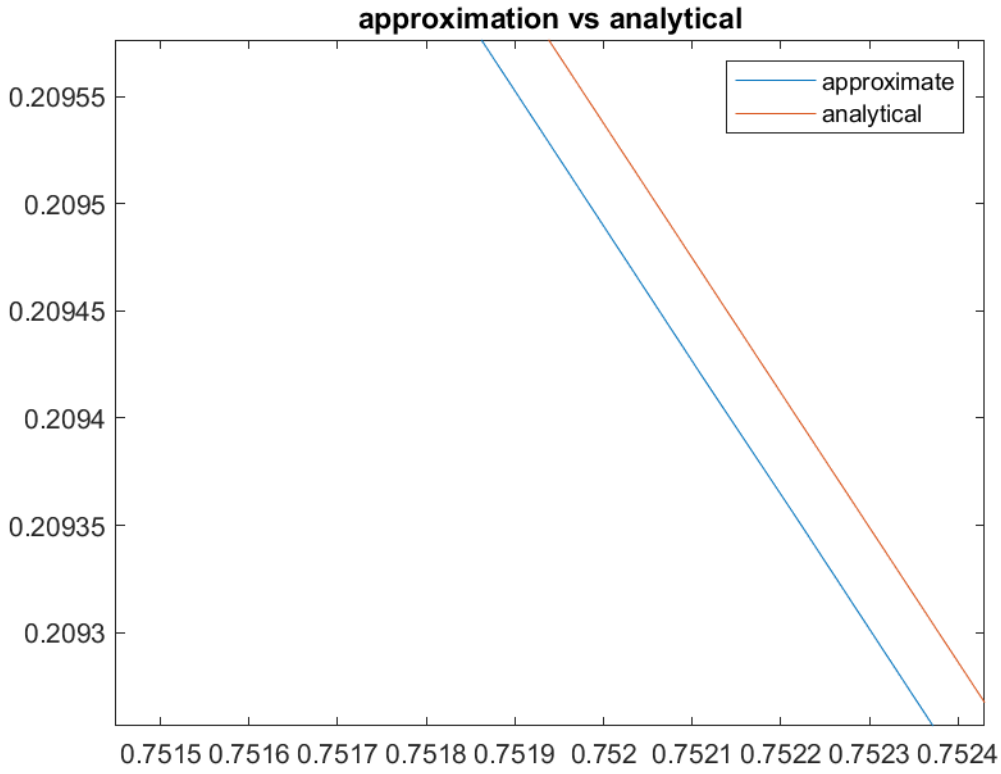


Figure 3: A close view of the approximate solution and the analytical solution

Viewing at the figure above we can see that the Δt being at .001 leads to a close approximation. Viewing the plot, we can see that at an instance of t there is a .00005 upward shift. Looking at figure 2 we can also see that to the naked eye that plot looks almost identical.

2) Problem two – third order differential

a) Find and plot the analytical solution to the differential equation for $0 \leq t \leq 10$. Identify the roots of the characteristic equation and plot them in the complex plane.

Work for solving the differential equation will be in appendix B and the code will be in appendix A

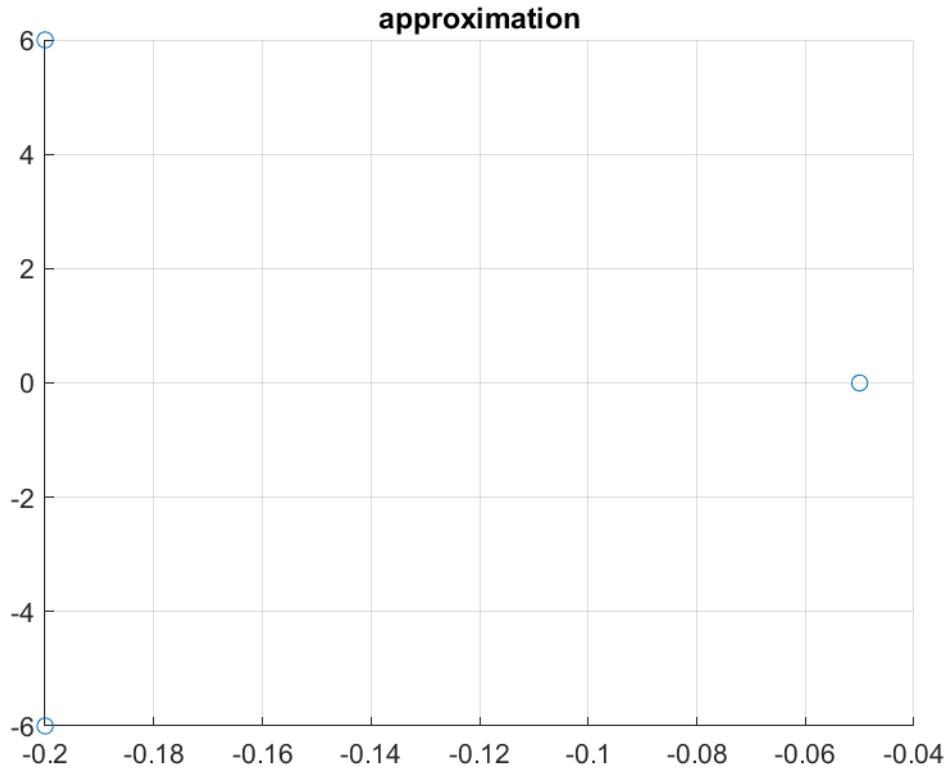


Figure 4: scatter plot of the roots of the differential equation using Matlab.

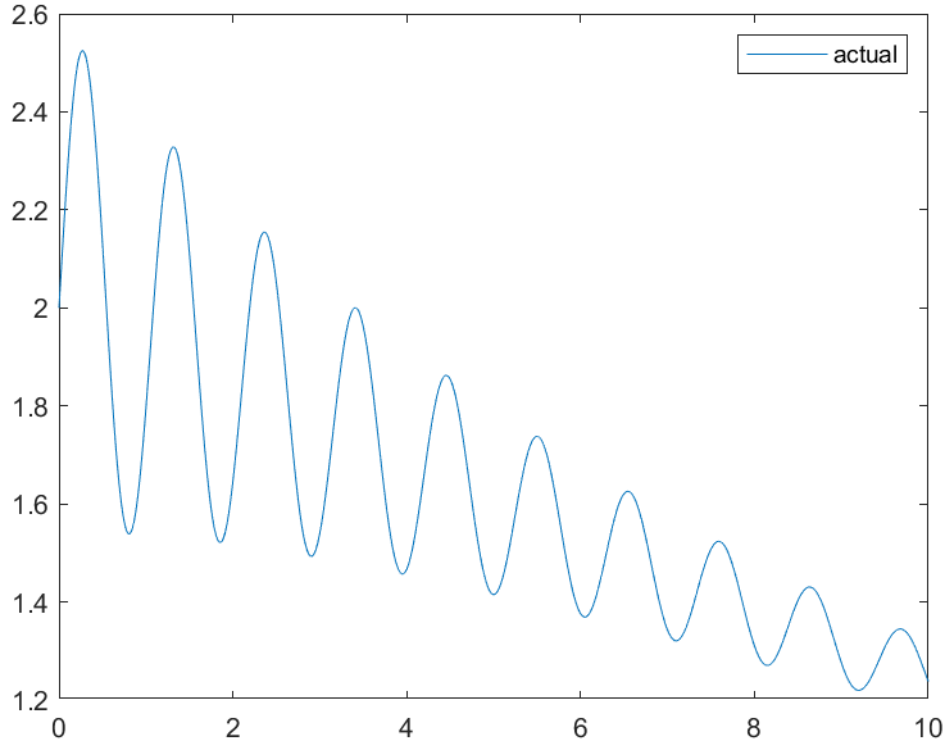


Figure 5: analytical solution plotted using Matlab.

b) Put the third-order differential equation into state-space form as described above.

$$I = \{\{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\}\}$$

$$a = \{\{0, 1, 0\}, \{0, 0, 1\}, \{-1.802, -36.06, -0.45\}\}$$

$$X(t) = \{2, 3, 1\}$$

$$A = a * \Delta t$$

$$A = \{\{0, .0001, 0\}, \{0, 0, .0001\}, \{-0.0001802, -0.003606, -0.000045\}\}$$

$$Temp = A + I$$

$$Temp = \{\{1, .0001, 0\}, \{0, 1, .0001\}, \{0.9998198, -0.003606, -0.000045\}\}$$

$$X_next = X_now * Temp$$

$$X_next = \{2.003, 3.0001, 1.9888\}$$

Note that this is only for the first iteration of the for loop. As the loop progresses the values of X_now and X_next will change but this is the general state space form of the differential equation.

c) Write a program in C(++) to plot an approximate solution using (8). You may have to try several values of Δt to get a reasonable approximation.

d) Compare the exact solution with the approximate solution

The program used to approximate the solution will be located in appendix A. The value of Δt that ended up with the close approximation was .0001. having values less than this lead to approximations that did not match the analytical solution.

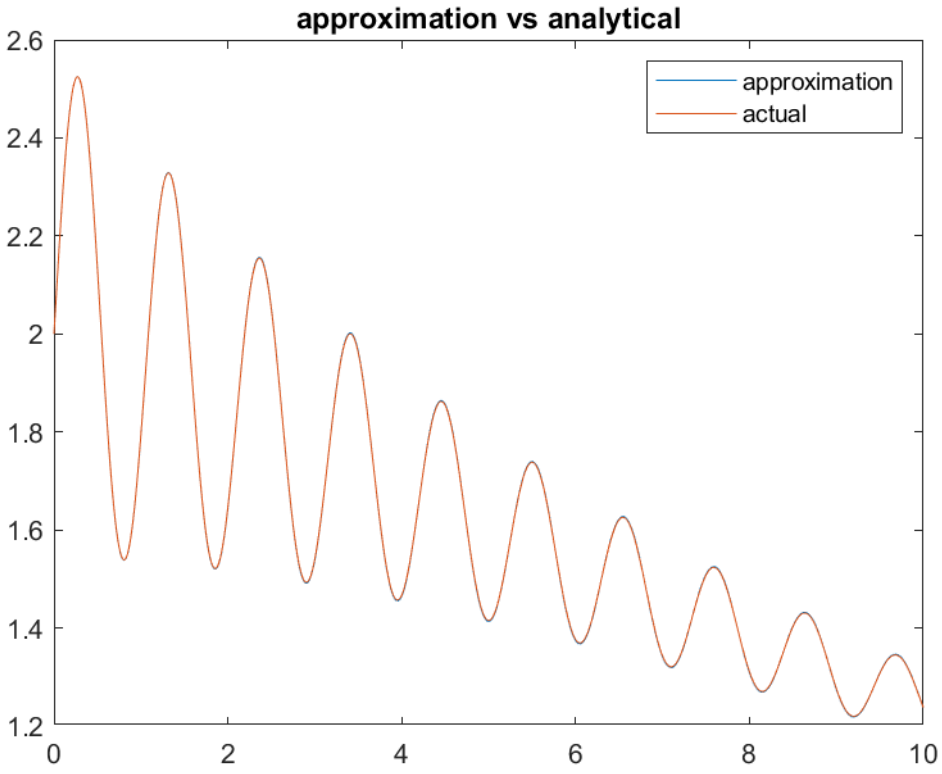


Figure 6: this is the plot of the analytical solution and the approximate solution using Matlab

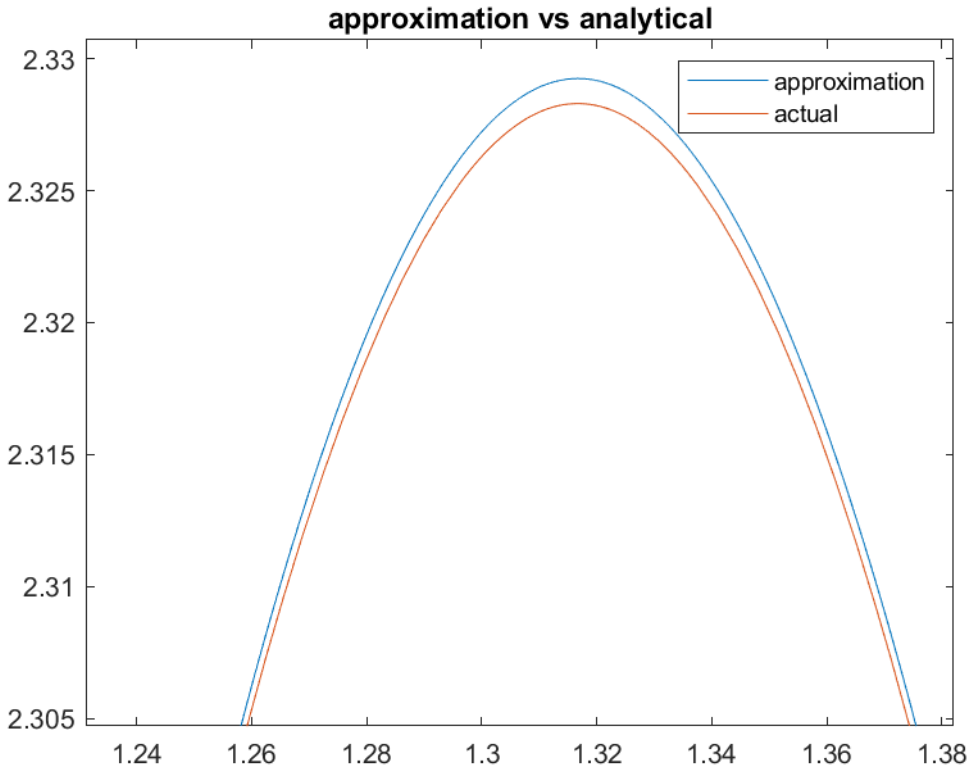


Figure 7: this is the zoomed in version to show the error of the approximation vs the analytical.

3) Problem three – second order differential

- ***a) Determine a differential equation relating the input f(t) to the output y(t).***
- ***b) Determine the initial condition for ˙y(t) if i1(0) = 0.2 A and y(0) = 5 V.***
- ***c) Determine the analytical solution for the zero-input response of the system with these initial conditions.***

Answer and work for (a) through (c) are in appendix B of this document.

d) Represent the differential equation for the circuit in state space form.

$$I = \{\{1, 0\}, \{0, 1\}\}$$

$$a = \{\{0, 1\}, \{-41.66, -196.04\}\}$$

$$X(t) = \{5, 40.625\}$$

$$A = a * \Delta t$$

$$A = \{\{0, .0001\}, \{-0.004166, -0.0196.04\}\}$$

Temp = A + I

Temp = {{1, .0001}, {-0.0005, - 0.9959375}}

X_next = X_now * Temp

X_next = {5.0041, 39.8078}

e) Using your program, determine a numerical solution to the differential equation for the zero-input response.

Code is in appendix A that shows the numerical solution to the differential equation.

f) Plot and compare the analytical and the numerical solution. Comment on your results.

As shown below using a small Δt (.0001) we can approximate the solution to the differential equation well. Noting Figure 8 there is some error in comparison of the two but small enough that it could be considered negligible.

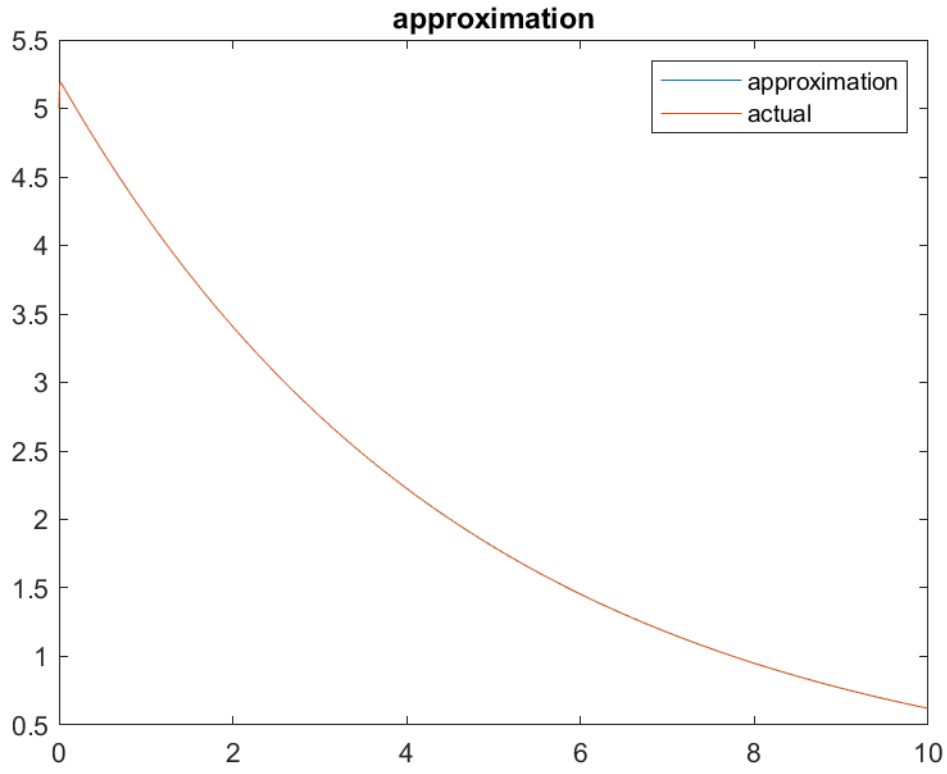


Figure 8: This is a plot with the analytical vs the approximation using Matlab

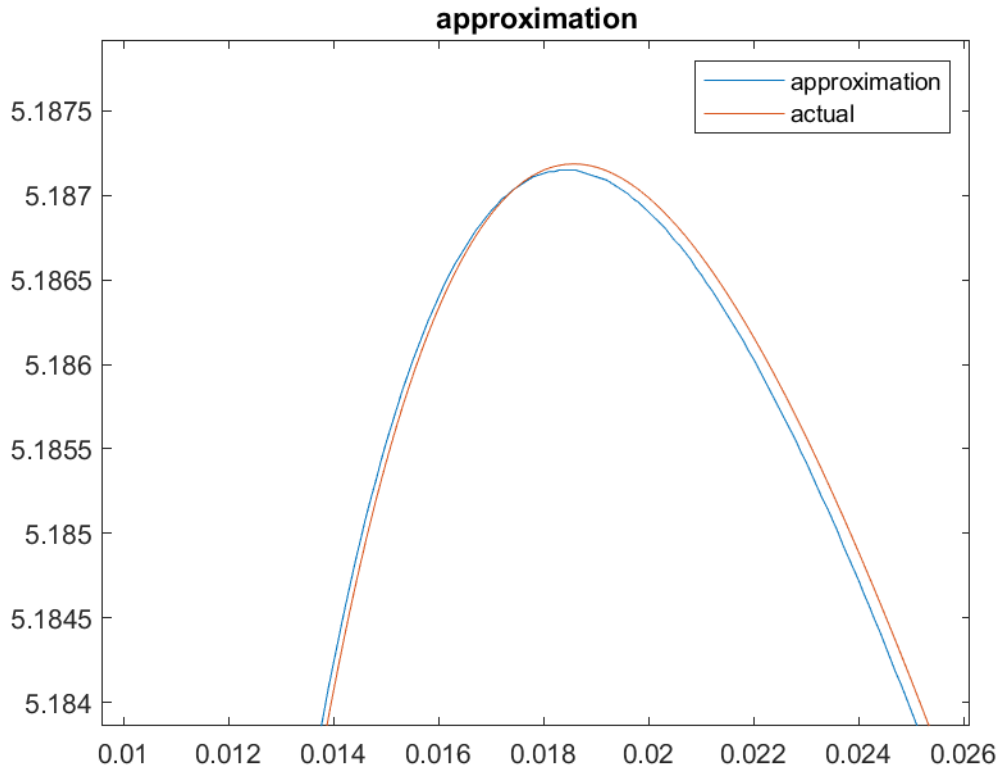


Figure 9: This is a zoomed in picture of Figure 7 showing the error in the approximation using Matlab.

g) Suppose that the circuit had nonlinear element in it, such as dependent sources. How would the analytical solution and numerical solution would change (in general)?

The methodology of approximating worked because it was a linear differential equation. If we added a non-linear element in the circuit, we would have to use a different method to approximate the solution.

Appendix A – Code snippets

First order differential code

```
/****************************************
Assignment: Numerical Solution of Differential Equations
: The Zero-Input Solution

pseudocode for first order equations

      DEFINE delta_t = time stepping


      get initial conditions
      get constant a (equation 1)

      set yo as initial conditions

      open output file

      create integers y_now and y_now_delta_t and t

      y_now = yo
      t = 0

      loop
            y_now_delta_t = (1 + a*delta_t)y_now
            store y_now in text file
            store t in text file

            t = t + delta_t
            y_now = y_now_delta_t

      while (counter < 10/delta_t)

      close output file



****************************************/



#include <fstream>
#include <iostream>
using namespace std;

int main(void){

      int a, yo;
      float t;
      float Delta_t = .0001;
```

```
	cout << "please enter in a: ";
	cin >> a;
	cout << "please enter in yo: ";
	cin >> yo;

	float y_now, y_next;
	a = -a;

	ofstream outfile("approximation");

	y_now = yo;

	for(t = 0; t < 10; t = t + Delta_t){
		y_next = (1 + a * Delta_t)*y_now;
		outfile << t << " " << y_now << endl;
		y_now = y_next;
	}

	outfile.close();

	return 0;
}
```

Third order differential code

```
/****************************************
Assignment: Numerical Solution of Differential Equations
:The Zero-Input Solution

pseudocode for third order equations

	DEFINE delta_t = time stepping


	get initial conditions
	get constant a0, a1, a2 (equation 1)

	set yo, yo', yo'' as initial conditions

	open output file

	create integers y_now and y_now_delta_t and t

	X = [yo, yo', yo'']
	t = 0

	loop
		y_now_delta_t = (1 + a*delta_t)y_now
		store y_now in text file
		store t in text file
```

```
            t = t + delta_t
            y_now = y_now_delta_t

      while (counter < 10/delta_t)

      close output file
**************************************/

#include <fstream>
#include <iostream>

using namespace std;

void mattimes(double t, double a[][2],double m[][2])
{
      int i,j;
      for(i = 0; i < 2; i++)
            for(j = 0; j < 2; j++)
                  m[i][j] = t*a[i][j];
}
void matadd(double a[][2],double b[][2],double c[][2])
{
      int i,j;
      for(i = 0; i < 2; i++)
            for(j = 0; j < 2; j++)
                  c[i][j] = a[i][j]+b[i][j];
}

void matvecmult(double m[][2], double *v, double *prod)
{
      double sum;
      int i,j;
      for(i = 0; i < 2; i++) {
            sum = 0;
            for(j = 0; j < 2; j++) {
                  sum += m[i][j]*v[j];
            }
      prod[i] = sum;
      }
}


int main(void){

      double yo, yoD, yoD2;
      double t = 0;
      double delta_t = .0001;
      double a0, a1, a2;
      double X_next[2] = {0};

      ofstream outfile("approximation");

      cout << "enter a0: ";
```

```
	cin >> a0;
	cout << "enter a1: ";
	cin >> a1;
	cout << "enter yo: ";
	cin >> yo;
	cout << "enter yo': ";
	cin >> yoD;

	double X_now[2] = {yo, yoD};

	double a[2][2] = {{0, 1}, {-a0, -a1}};
	double I[2][2] = {{1, 0}, {0, 1}};
	double A[2][2] = {0};
	double temp[2][2] = {0};
	double *Prod1;
	double Prod2[2][2] = {0};
	mattimes(delta_t, a, A);

	for(t = 0; t < 10; t = t + delta_t){
		matadd(A, I, temp);
		matvecmult(temp, X_now, X_next);
		outfile << t << " " << X_now[0] << endl;
		for(int i = 0; i < 2; i++){
			X_now[i] = X_next[i];
		}
	}

	outfile.close();

	return 0;
}
```

Second order differential code

```
/****************************************
Assignment: Numerical Solution of Differential Equations
:The Zero-Input Solution

pseudocode for second order equations

	DEFINE delta_t = time stepping


	get initial conditions
	get constant a0, a1 (equation 1)

	set yo, yo' as initial conditions

	open output file

	create integers y_now and y_now_delta_t and t
```

```
      X = [yo, yo']
      t = 0

      loop
            y_now_delta_t = (1 + a*delta_t)y_now
            store y_now in text file
            store t in text file

            t = t + delta_t
            y_now = y_now_delta_t

      while (counter < 10/delta_t)

      close output file
****************************************/

#include <fstream>
#include <iostream>

using namespace std;

void mattimes(double t, double a[][2],double m[][2])
{
      int i,j;
      for(i = 0; i < 2; i++)
            for(j = 0; j < 2; j++)
                  m[i][j] = t*a[i][j];
}
void matadd(double a[][2],double b[][2],double c[][2])
{
      int i,j;
      for(i = 0; i < 2; i++)
            for(j = 0; j < 2; j++)
                  c[i][j] = a[i][j]+b[i][j];
}

void matvecmult(double m[][2], double *v, double *prod)
{
      double sum;
      int i,j;
      for(i = 0; i < 2; i++) {
            sum = 0;
            for(j = 0; j < 2; j++) {
                  sum += m[i][j]*v[j];
            }
      prod[i] = sum;
      }
}


int main(void){

      double yo, yoD, yoD2;
      double t = 0;
```

```
	double delta_t = .0001;
	double a0, a1, a2;
	double X_next[2] = {0};

	ofstream outfile("approximation");

	cout << "enter a0: ";
	cin >> a0;
	cout << "enter a1: ";
	cin >> a1;
	cout << "enter yo: ";
	cin >> yo;
	cout << "enter yo': ";
	cin >> yoD;

	double X_now[2] = {yo, yoD};

	double a[2][2] = {{0, 1}, {-a0, -a1}};
	double I[2][2] = {{1, 0}, {0, 1}};
	double A[2][2] = {0};
	double temp[2][2] = {0};
	double *Prod1;
	double Prod2[2][2] = {0};
	mattimes(delta_t, a, A);

	for(t = 0; t < 10; t = t + delta_t){
		matadd(A, I, temp);
		matvecmult(temp, X_now, X_next);
		outfile << t << " " << X_now[0] << endl;
		for(int i = 0; i < 2; i++){
			X_now[i] = X_next[i];
		}
	}

	outfile.close();

	return 0;
}
```

Appendix – B